



COURSE CODE/TITLE: CPE 010

SECTION: CPE21S1

DATE PERFORMED: 08-07-26

DATE SUBMITTED: 08-07-26

NAME: Paula Esguerra

INSTRUCTOR: Engr. Jimlord Quejado

HOA NO. 5 Hands-on Activity 5.1 Queues

1. PROCEDURE OUTPUT

QUEUE

```
1  #include <iostream>
2  #include "queueArr.h"
3
4  int main() {
5      //creating an object
6      std::cout << "----- \n";
7      std::cout << " Testing Initialization and Enqueue \n";
8      std::cout << "----- \n";
9      queueArr<int> q(5);
10
11     std::cout<<std::endl;
12     std::cout << "Is my queue empty? " << (q.Empty() ? "yes" : "no") << std::endl;
13     std::cout<<std::endl;
14
15     q.Enqueue(10);
16     std::cout << "Queue Size: " << q.Size() << std::endl;
17     std::cout << "Is my queue full? " << (q.Full() ? "yes" : "no" ) <<std::endl;
18     std::cout << "Front Element: " << q.Front() << std::endl;
19     std::cout << "Back Element: " << q.Back() << std::endl;
20     std::cout<<std::endl;
21
22     q.Enqueue(20);
23     std::cout << "Queue Size: " << q.Size() << std::endl;
24     std::cout << "Is my queue full? " << (q.Full() ? "yes" : "no" ) <<std::endl;
25     std::cout << "Front Element: " << q.Front() << std::endl;
26     std::cout << "Back Element: " << q.Back() << std::endl;
27     std::cout<<std::endl;
28
29     q.Enqueue(30);
30     std::cout << "Queue Size: " << q.Size() << std::endl;
31     std::cout << "Is my queue full? " << (q.Full() ? "yes" : "no" ) <<std::endl;
32     std::cout << "Front Element: " << q.Front() << std::endl;
33     std::cout << "Back Element: " << q.Back() << std::endl;
34     std::cout<<std::endl;
35
36     q.Enqueue(40);
37     std::cout << "Queue Size: " << q.Size() << std::endl;
38     std::cout << "Is my queue full? " << (q.Full() ? "yes" : "no" ) <<std::endl;
39     std::cout << "Front Element: " << q.Front() << std::endl;
40     std::cout << "Back Element: " << q.Back() << std::endl;
41     std::cout<<std::endl;
42 }
```

```

43 q.Enqueue(50);
44 std::cout << "Queue Size: " << q.Size() << std::endl;
45 std::cout << "Is my queue full?" << (q.Full() ? "yes" : "no") << std::endl;
46 std::cout << "Front Element: " << q.Front() << std::endl;
47 std::cout << "Back Element: " << q.Back() << std::endl;
48 std::cout << std::endl;
49
50 std::cout << "----- \n";
51 std::cout << " Testing Overflow Trigger \n";
52 std::cout << "----- \n";
53 q.Enqueue(60);
54 std::cout << std::endl;
55 std::cout << std::endl;
56
57 std::cout << "----- \n";
58 std::cout << " Testing Dequeue \n";
59 std::cout << "----- \n";
60 std::cout << "Dequeue: " << q.Dequeue() << std::endl;
61 std::cout << "Dequeue: " << q.Dequeue() << std::endl;
62 std::cout << "New Front Element: " << q.Front() << std::endl;
63 std::cout << std::endl;
64 std::cout << std::endl;
65
66 std::cout << "----- \n";
67 std::cout << " Testing Circular \n";
68 std::cout << "----- \n";
69 q.Enqueue(70);
70 q.Enqueue(80);
71 std::cout << "New Back Element: " << q.Back() << std::endl;
72 std::cout << "Current Size: " << q.Size() << std::endl;
73 std::cout << std::endl;
74 std::cout << std::endl;
75
76 std::cout << "----- \n";
77 std::cout << " Testing Clear and Underflow \n";
78 std::cout << "----- \n";
79 q.Clear();
80 std::cout << "Size after clear: " << q.Size() << std::endl;
81 q.Dequeue();
82 std::cout << std::endl;
83 std::cout << std::endl;
84
85 std::cout << "----- \n";
86 std::cout << " Testing Copy Constructor \n";
87 std::cout << "----- \n";
88 queueArr<int> qA(5);
89 qA.Enqueue(100);
90 qA.Enqueue(200);
91
92 queueArr<int> qB = qA;
93 std::cout << "Queue B front: " << qB.Front() << std::endl;
94
95 qA.Dequeue();
96 std::cout << "After modifying the front is still: " << qB.Front() << std::endl;
97 std::cout << std::endl;
98 std::cout << std::endl;
99
100 std::cout << "----- \n";
101 std::cout << " END \n";
102 std::cout << "----- \n";
103 return 0;
104 }

```

HEADER QUEUE:

```

1  #ifndef QUEUEARR_H
2  #define QUEUEARR_H
3
4  #include <iostream>
5
6  //array-based circular queue
7  template <typename T>
8  class queueArr{
9      private:
10         //pointer to dynamically allocate the array
11         T*q_array;
12
13         //maximum number of elements of queue can hold
14         size_t q_capacity;
15
16         // current number of the elements in the wueue
17         size_t q_size;
18
19         //index of the front element
20         int q_front;
21
22         // indec of the back element
23         int q_back;
24
25     public:
26
27         //constructor
28         queueArr(size_t capacity = 10);
29
30         //copy constructor
31         queueArr(const queueArr& other);
32
33         //copy assignment operator
34         queueArr& operator=(const queueArr& other);
35
36         //destructor
37         ~queueArr();
38
39         //queue operations
40         bool Empty();
41         bool Full();
42

```

```

43         size_t Size();
44
45         void Clear();
46
47         T Front();
48
49         T Back();
50
51         void Enqueue(T value);
52
53         T Dequeue();
54     };
55
56     //constructor
57     template <typename T>
58     queueArr<T>::queueArr(size_t capacity) {
59         q_capacity = capacity;
60         q_size = 0;
61         q_front = 0;
62         q_back = -1;
63         q_array = new T[q_capacity];
64     }
65
66
67     //copy constructor
68     template <typename T>
69     queueArr<T>::queueArr(const queueArr& other) {
70         q_capacity = other.q_capacity;
71         q_size = other.q_size;
72         q_front = other.q_front;
73         q_back = other.q_back;
74
75         // Allocate brand new array memory
76         q_array = new T[q_capacity];
77
78         // Perform a deep copy of elements
79         for (size_t i = 0; i < q_capacity; i++) {
80             q_array[i] = other.q_array[i];
81         }
82     }
83
84
85     //copy assignment operator
86     template <typename T>
87     queueArr<T>& queueArr<T>::operator=(const queueArr& other) {
88         // 1. Guard against self-assignment (e.g., q1 = q1)
89         if (this != &other) {
90             // 2. Free current existing memory
91             delete[] q_array;
92
93             // 3. Copy scalar values
94             q_capacity = other.q_capacity;
95             q_size = other.q_size;
96             q_front = other.q_front;
97             q_back = other.q_back;
98
99             // 4. Allocate and copy array elements
100            q_array = new T[q_capacity];
101            for (size_t i = 0; i < q_capacity; i++) {
102                q_array[i] = other.q_array[i];
103            }
104        }
105        return *this; // Return reference to self

```

```

106 }
107
108 //destructor
109 template <typename T>
110 queueArr<T>::~queueArr() {
111     delete[] q_array; // Free dynamic memory allocation
112 }
113
114 //queue operations
115
116 //empty()
117 template<typename T>
118 bool queueArr<T>::Empty() {
119     //check if the size is 0
120     return q_size == 0 ;
121 }
122
123 //Full()
124 template<typename T>
125 bool queueArr<T>::Full() {
126     //check if the q_size reached the maximum capacity
127     return q_size == q_capacity;
128 }
129
130
131 //sizeT
132 template <typename T>
133 size_t queueArr<T>::Size() { //identifier "size_t" is undefined
134     return q_size;
135 }
136
137 //clear;
138 template<typename T>
139 T queueArr<T>::Front() {
140     // check if the queue is empty
141     if(Empty()) {
142         std::cout<<"Queue is Empty\n";
143         return T{};
144     }
145
146     //return the front
147     return q_array[q_front];

```

```

147     return q_array[q_front];
148
149 }
150
151 //back
152 template <typename T>
153 T queueArr<T>::Back() {
154     //check if the queue is empty
155     if (Empty()) {
156         std::cout << "queue is empty" << std::endl;
157         return T{};
158     }
159     return q_array[q_back];
160 }
161
162 template <typename T>
163 void queueArr<T>::Clear() {
164     //reset the q_size, q_front, q_back
165     q_size = 0;
166     q_front = 0;
167     q_back = -1;
168 }
169
170 template <typename T>
171 void queueArr<T>::Enqueue(T value) {
172     // check if the queue is Full
173     if (Full()) {
174         std::cout << "Queue is Full\n";
175         return;
176     }
177
178     // if inserting the very first element, set q_front to 0
179     if (Empty()) {
180         q_front = 0;
181     }
182
183     // move q_back circularly
184     q_back = (q_back + 1) % q_capacity;
185 }

```

```

187 // move q_back circularly
188 q_back = (q_back + 1) % q_capacity;
189
190 // store the value to the back
191 q_array[q_back] = value;
192
193 // increment the q_size
194 q_size++;
195 }
196
197 //T dequeue();
198 template<typename T>
199 T queueArr<T>::Dequeue() {
200     // check if ifempty
201     if (Empty()) {
202         std::cout << "Queue is Empty\n";
203         return T{};
204     }
205
206     //create a temporary variable to store the current front
207     T temp = q_array[q_front];
208
209     //move q_frint circularly
210     q_front = (q_front + 1) % q_capacity;
211
212     //decrement the q_size
213     q_size--;
214
215     //reset the indexes of the queue become empty
216     if(Empty()) Clear();
217
218     //return the temporary variable
219     return temp;
220 }
221
222 }
223
224 #endif //QUEUEARR_H

```

OUTPUT QUEUE:

```
D:\CPE 010\HOA#6\main.exe x + v
-----
Testing Initialization and Enqueue
-----
Is my queue empty? yes

Queue Size: 1
Is my queue full? no
Front Element: 10
Back Element: 10

Queue Size: 2
Is my queue full? no
Front Element: 10
Back Element: 20

Queue Size: 3
Is my queue full? no
Front Element: 10
Back Element: 30

Queue Size: 4
Is my queue full? no
Front Element: 10
Back Element: 40

Queue Size: 5
Is my queue full? yes
Front Element: 10
Back Element: 50

-----
Testing Overflow Trigger
-----
Queue is Full
```



```
D:\CPE 010\HOA#6\main.exe × + ▾
-----
Testing Dequeue
-----
Dequeue: 10
Dequeue: 20
New Front Element: 30

-----

Testing Circular
-----
New Back Element: 80
Current Size: 5

-----

Testing Clear and Underflow
-----
Size after clear: 0
Queue is Empty

-----

Testing Copy Constructor
-----
Queue B front: 100
After modifying the front is still: 100

-----

END
-----

Process exited after 0.3525 seconds with return value 0
Press any key to continue . . . |
```

QUEUE PRINTER:

```

1  #ifndef PRINTER_JOB_H
2  #define PRINTER_JOB_H
3
4  #include <iostream>
5  #include <string>
6
7  class Job {
8      private:
9          int jobId;
10         std::string userName;
11         int numPages;
12
13     public:
14         // FIXED: Corrected member names in the initializer list
15         Job() : jobId(0), userName(""), numPages(0) {}
16
17         Job(int id, std::string name, int pages) {
18             jobId = id;
19             userName = name;
20             numPages = pages;
21         }
22
23         void displayJobDetails() const {
24             std::cout << "[Job ID: " << jobId << " | User: " << userName
25                 << " | Pages: " << numPages << "]" << "\n";
26         }
27
28         int getPages() const { return numPages; }
29     };
30
31     // FIXED: Added missing opening brace ' {
32     class Printer {
33     private:
34         static const int maxCap = 10;
35         Job queue[maxCap];
36         int front;
37         int rear;
38         int size;
39
40     public:
41         Printer() {
42             front = 0;

```

```

43     rear = -1;
44     size = 0;
45 }
46
47 bool isFull() const {
48     return size == maxCap;
49 }
50
51 bool isEmpty() const {
52     return size == 0;
53 }
54
55 void addJob(const Job& job) {
56     if (isFull()) {
57         std::cout << "Printer queue is full\n";
58         return;
59     }
60
61     rear = (rear + 1) % maxCap;
62     queue[rear] = job;
63     size++;
64
65     std::cout << "Added to queue: ";
66     job.displayJobDetails();
67     std::cout << std::endl;
68 }
69
70 void processJob() {
71     if (isEmpty()) {
72         std::cout << "No pending jobs\n";
73         return;
74     }
75
76     Job currentJob = queue[front];
77
78     front = (front + 1) % maxCap;
79     size--;
80
81     std::cout << "Printing in process: ";
82     currentJob.displayJobDetails();
83     std::cout << "-> Successfully Printed\n";
84     currentJob.displayJobDetails();
85     std::cout << "-> Successfully Printed\n";
86 }
87 };
88 #endif
89

```

```

1  #include <iostream>
2  #include <string>
3  #include "printer_job.h"
4
5  int main() {
6      Printer p;
7
8      std::cout << "----- \n";
9      std::cout << "Simulating Job Additions\n";
10     std::cout << "----- \n";
11
12     p.addJob( Job (107, "Paul", 9));
13     p.addJob( Job (108, "Sarah", 2));
14     p.addJob( Job (109, "Rade", 4));
15     std::cout << std::endl;
16     std::cout << std::endl;
17
18     std::cout << "----- \n";
19     std::cout << "Simulating Printer Processing\n";
20     std::cout << "----- \n";
21
22     p.processJob();
23     p.processJob();
24     std::cout << std::endl;
25     std::cout << std::endl;
26
27     std::cout << "----- \n";
28     std::cout << "Adding more assignment\n";
29     std::cout << "----- \n";
30     p.addJob( Job (110, "Diana", 3));
31     std::cout << std::endl;
32     std::cout << std::endl;
33
34     std::cout << "----- \n";
35     std::cout << "Processing Remaining Job\n";
36     std::cout << "----- \n";
37     p.processJob();
38     p.processJob();
39     p.processJob();
40     std::cout << std::endl;
41     std::cout << std::endl;
42
43     return 0;
44 }
45

```

OUTPUT PRINTER:

```
D:\CPE 010\hoa na hindi na al: × + v

-----
Simulating Job Additions
-----
Added to queue: [Job ID: 107 | User: Paul | Pages: 9]
Added to queue: [Job ID: 108 | User: Sarah | Pages: 2]
Added to queue: [Job ID: 109 | User: Rade | Pages: 4]

-----
Simulating Printer Processing
-----
Printing in process: [Job ID: 107 | User: Paul | Pages: 9] -> Successfully Printed
Printing in process: [Job ID: 108 | User: Sarah | Pages: 2] -> Successfully Printed

-----
Adding more assignment
-----
Added to queue: [Job ID: 110 | User: Diana | Pages: 3]

-----
Processing Remaining Job
-----
Printing in process: [Job ID: 109 | User: Rade | Pages: 4] -> Successfully Printed
Printing in process: [Job ID: 110 | User: Diana | Pages: 3] -> Successfully Printed
No pending jobs
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

QUEUE PRINTER:

```

1  #ifndef PRINTER_JOB_H
2  #define PRINTER_JOB_H
3
4  #include <iostream>
5  #include <string>
6
7  class Job {
8      private:
9          int jobId;
10         std::string userName;
11         int numPages;
12
13     public:
14         // FIXED: Corrected member names in the initializer list
15         Job() : jobId(0), userName(""), numPages(0) {}
16
17         Job(int id, std::string name, int pages) {
18             jobId = id;
19             userName = name;
20             numPages = pages;
21         }
22
23         void displayJobDetails() const {
24             std::cout << "[Job ID: " << jobId << " | User: " << userName
25                 << " | Pages: " << numPages << "]" << "\n";
26         }
27
28         int getPages() const { return numPages; }
29     };
30
31     // FIXED: Added missing opening brace '{'
32     class Printer {
33     private:
34         static const int maxCap = 10;
35         Job queue[maxCap];
36         int front;
37         int rear;
38         int size;
39
40     public:
41         Printer() {
42             front = 0;

```

```

43     rear = -1;
44     size = 0;
45 }
46
47 bool isFull() const {
48     return size == maxCap;
49 }
50
51 bool isEmpty() const {
52     return size == 0;
53 }
54
55 void addJob(const Job& job) {
56     if (isFull()) {
57         std::cout << "Printer queue is full\n";
58         return;
59     }
60
61     rear = (rear + 1) % maxCap;
62     queue[rear] = job;
63     size++;
64
65     std::cout << "Added to queue: ";
66     job.displayJobDetails();
67     std::cout << std::endl;
68 }
69
70 void processJob() {
71     if (isEmpty()) {
72         std::cout << "No pending jobs\n";
73         return;
74     }
75
76     Job currentJob = queue[front];
77
78     front = (front + 1) % maxCap;
79     size--;
80
81     std::cout << "Printing in process: ";
82     currentJob.displayJobDetails();
83     std::cout << "-> Successfully Printed\n";
84     currentJob.displayJobDetails();
85     std::cout << "-> Successfully Printed\n";
86 }
87 };
88 #endif
89

```

```

1  #include <iostream>
2  #include <string>
3  #include "printer_job.h"
4
5  int main() {
6      Printer p;
7
8      std::cout << "----- \n";
9      std::cout << "Simulating Job Additions\n";
10     std::cout << "----- \n";
11
12     p.addJob( Job (107, "Paul", 9));
13     p.addJob( Job (108, "Sarah", 2));
14     p.addJob( Job (109, "Rade", 4));
15     std::cout << std::endl;
16     std::cout << std::endl;
17
18     std::cout << "----- \n";
19     std::cout << "Simulating Printer Processing\n";
20     std::cout << "----- \n";
21
22     p.processJob();
23     p.processJob();
24     std::cout << std::endl;
25     std::cout << std::endl;
26
27     std::cout << "----- \n";
28     std::cout << "Adding more assignment\n";
29     std::cout << "----- \n";
30     p.addJob( Job (110, "Diana", 3));
31     std::cout << std::endl;
32     std::cout << std::endl;
33
34     std::cout << "----- \n";
35     std::cout << "Processing Remaining Job\n";
36     std::cout << "----- \n";
37     p.processJob();
38     p.processJob();
39     p.processJob();
40     std::cout << std::endl;
41     std::cout << std::endl;
42
43     return 0;
44 }
45

```


OUTPUT PRINTER:

```

D:\CPE 010\hoa na hindi na al. x + v
-----
Simulating Job Additions
-----
Added to queue: [Job ID: 107 | User: Paul | Pages: 9]
Added to queue: [Job ID: 108 | User: Sarah | Pages: 2]
Added to queue: [Job ID: 109 | User: Rade | Pages: 4]

-----
Simulating Printer Processing
-----
Printing in process: [Job ID: 107 | User: Paul | Pages: 9] -> Successfully Printed
Printing in process: [Job ID: 108 | User: Sarah | Pages: 2] -> Successfully Printed

-----
Adding more assignment
-----
Added to queue: [Job ID: 110 | User: Diana | Pages: 3]

-----
Processing Remaining Job
-----
Printing in process: [Job ID: 109 | User: Rade | Pages: 4] -> Successfully Printed
Printing in process: [Job ID: 110 | User: Diana | Pages: 3] -> Successfully Printed
No pending jobs

```

CONCLUSION

Through this activity, I have learned how to implement a queue function using a circular array based, the procedure instructed how to fixed-size contiguous block of memory that can efficiently manage the arriving data without having a memory fragmentation or dynamic allocation through pointer-based linked structures. By using modular arithmetic to advance the front and rear indices, this effectively resysled array postion, and optimizing system constraints to model hardware data. The supplementary on the other hand, manually writing the enqueue and dequeue operations gave the role of boundary conditions, avoiding array indices overflow and underflow while maintaining accurate size of the current element. Reflecting on my work, I tried to fulfill the procedures and try to keep up with the code, however my improvement is still laking the deep understand of the code, I need to practice more if I can and if I do I will be able to keep up in class and do much better performance.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Error assistance, and asking questions

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I pasted my work to check something and if there were an error, I will retype and try to understand the changed code.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.